You said:

exec(2).pyc
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 53s

ChatGPT said:

Análise de malware — `exec(2).pyc`

VEREDITO: MALWARE

Confiança alta — análise estática do bytecode e do código oculto

Categoria principal: Trojan Downloader / Loader (cavalo de Troia carregador de segundo estágio).

Comportamentos identificados: código ofuscado, comunicação com infraestrutura remota por meio da blockchain Solana, evasão baseada na localização e download e execução de código JavaScript externo.

1. Identificação do arquivo

O arquivo anexado foi examinado sem executar o programa.

Propriedade	Resultado
Nome	`exec(2).pyc`
Formato	Bytecode compilado de Python
Versão identificada	CPython 3.13
Tamanho	7.001 bytes
SHA-256	`32aa84af35da78e1e8707e7d93fd48873e30d4f559d514a2648c07e43d35775e`

O formato `.pyc` não é, por si só, indício de malware. A classificação depende do comportamento do código contido nele.

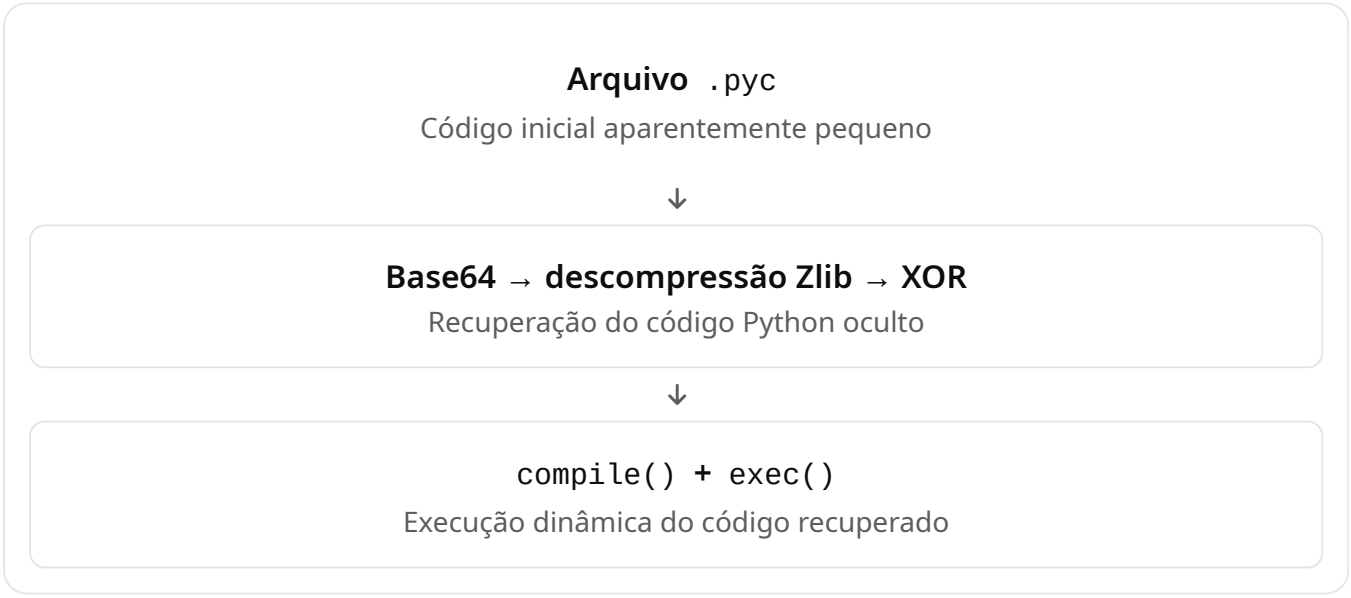
2. Ofuscação e ocultação do código

A primeira etapa revela uma camada de proteção destinada a esconder a lógica real do programa.

O módulo utiliza:

- `base64`, para decodificar uma sequência de dados;
- `zlib`, para descompactar o conteúdo;
- uma operação XOR com o valor `134` (`0x86`), para transformar os bytes;
- `compile()` e `exec()`, para compilar e executar o conteúdo recuperado.

O fluxo identificado é:



A transformação foi reproduzida estaticamente, sem executar o conteúdo resultante. Ela revelou um programa Python de aproximadamente 15,5 KB, com funcionalidades suspeitas descritas nas próximas etapas.

Avaliação: a combinação de ofuscação com execução dinâmica é um forte indicador de ocultação deliberada, embora não seja, isoladamente, prova definitiva de malware.

3. Comunicação com a blockchain Solana

O código recuperado define uma função que consulta o método JSON-RPC `getSignaturesForAddress` em diversos endpoints relacionados à rede Solana.

Um endereço específico aparece no código:

```
BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC
```

O programa procura transações que contenham um campo `memo`, extrai seu conteúdo e tenta interpretá-lo como JSON. Caso ainda não encontre os dados esperados, repete as consultas.

Essa lógica é particularmente suspeita porque o programa utiliza dados publicados na blockchain para descobrir instruções e informações sobre o próximo estágio de execução.

O JSON pode conter um campo chamado `link`, cujo valor é decodificado de Base64 para obter um endereço de rede.

Avaliação: comportamento compatível com um mecanismo de comando e controle (C2) ou de descoberta de infraestrutura maliciosa usando a blockchain como canal de comunicação indireto.

4. Evasão baseada no idioma e na localização

O código contém a função `_isRussianSystem()`, que verifica informações como:

- idioma e localidade do sistema;
- variáveis de ambiente;
- fuso horário;
- determinadas regiões e deslocamentos UTC.

Se os critérios associados a um sistema russo forem satisfeitos, a execução é interrompida.

O programa também aguarda dez segundos antes de começar a atividade principal.

Avaliação: a combinação de identificação regional e interrupção seletiva da execução é compatível com técnicas de evasão utilizadas por malwares para evitar determinados ambientes de análise ou vítimas em regiões específicas.

5. Controle de execução por arquivo local

O código examina um arquivo chamado `init.json` no diretório pessoal do usuário.

Ele verifica se existe uma data registrada e calcula um período de validade de dois dias. Quando essa validade ainda está vigente, o programa encerra sua execução antecipadamente.

No macOS, o código também cria ou atualiza esse arquivo com uma data.

Esse mecanismo funciona como um controle de frequência ou período de execução. Por si só, não comprova persistência automática após reinicializações, mas contribui para o controle do comportamento do programa ao longo do tempo.

6. Download e execução de código externo

Esta é a evidência mais forte de comportamento malicioso.

A função `download_nodejs()` obtém uma versão do Node.js conforme o sistema operacional e a arquitetura da máquina, instala os arquivos no diretório pessoal e tenta remover o arquivo compactado após a extração.

Em seguida, o código:

1. Obtém um endereço remoto a partir dos dados recuperados.
2. Faz uma requisição HTTP para esse endereço.
3. Lê o corpo da resposta e os cabeçalhos `ivbase64` e `secretkey`.
4. Monta um arquivo JavaScript chamado `i.js`.
5. Insere no JavaScript os valores recebidos remotamente.
6. Utiliza `atob()` e `eval()` no código gerado para executar conteúdo dinâmico.
7. Inicia o Node.js em segundo plano, ocultando a saída padrão e as mensagens de erro.

O trecho construído pelo programa contém, entre outros elementos:

```
const secretKey = '...';
const _iv = Buffer.from('...', "base64");
eval(atob('...'));
```

Os valores entre aspas são preenchidos com dados recebidos pela rede. Portanto, o comportamento da segunda etapa depende do conteúdo remoto e não está inteiramente contido no arquivo analisado.

Avaliação: o arquivo não se limita a consultar dados ou executar uma tarefa local. Ele prepara o ambiente, busca conteúdo remoto e o executa dinamicamente, característica central de um *downloader/loader* malicioso.

7. Resumo das evidências

Comportamento observado	Interpretação

Payload oculto com Base64, Zlib e XOR	Ofuscação
Uso de `compile()` e `exec()`	Execução dinâmica do código oculto
Consultas à blockchain Solana e leitura de `memo`	Descoberta remota de configuração ou instruções
Bloqueio baseado em idioma e localização	Evasão seletiva
Controle temporal por `init.json`	Limitação ou controle da execução
Download de componente e criação de `i.js`	Preparação de um segundo estágio
Execução de JavaScript recebido remotamente	Carregamento e execução de código externo

8. Classificação final

Arquivo classificado como malware

Categoria principal: Trojan Downloader / Loader.

Justificativa: o programa combina código deliberadamente ofuscado, descoberta de instruções por meio da blockchain, evasão seletiva e execução de conteúdo remoto. O conjunto desses comportamentos sustenta a classificação maliciosa com alto grau de confiança.

Limitação importante

A análise permite classificar o arquivo como um carregador malicioso, mas não determinar com segurança a finalidade exata do payload final. O código recuperado não demonstra, por si só, que houve roubo de senhas, exfiltração de arquivos ou furto de carteiras de criptomoedas. Essas ações podem estar no segundo estágio, que é obtido remotamente e não foi executado nem baixado durante esta análise.

Recomendação: não execute o arquivo. Se ele já tiver sido executado em uma máquina real, trate o sistema como potencialmente comprometido e investigue os processos Node.js, o arquivo `i.js`, eventuais alterações no diretório pessoal e as conexões de rede associadas.

Também posso criar ↪ Criar diagrama do malware para facilitar a visualização da cadeia de ataque.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Trojan Downloader/Loader (cavalo de Troia carregador de segundo estágio)**.

Ele recupera instruções remotas, baixa componentes adicionais e executa código JavaScript dinamicamente. A comunicação por meio da blockchain Solana e as técnicas de evasão regional reforçam essa classificação.